

**Chosen database:** PostgreSQL 16, co-located with consumer-v3 on a t3.micro EC2 instance.  
**PostgreSQL-specific features used:**

| Criterion              | PostgreSQL                                                                           | Alternatives                                                                                |
|------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Bulk insert API        | Native <code>COPY FROM</code> binary protocol — 3–5× faster than <code>INSERT</code> | MySQL: no native <code>COPY</code> ; Cassandra: batch writes only                           |
| Upsert                 | <code>ON CONFLICT DO UPDATE</code> — idempotent per-key increment                    | MongoDB: <code>\$inc</code> upsert available; MySQL: <code>ON DUPLICATE KEY</code> (weaker) |
| Time-range queries     | <code>TIMESTAMPTZ</code> , B-tree range scans, <code>date_trunc</code> bucketing     | Redis: no durable history; DynamoDB: requires sort key design upfront                       |
| ACID compliance        | Full — no message loss on crash or retry                                             | Cassandra: eventual consistency; tunable but complex                                        |
| Operational simplicity | Single binary, <code>yum install postgresql16</code>                                 | Cassandra: cluster setup even for one node                                                  |

**Binary `COPY` protocol (pgx CopyFrom):**

The pgx driver's CopyFrom uses PostgreSQL's binary wire format, bypassing per-row SQL parsing and executor overhead. At 5,000 rows/batch this sustains ~150K–200K msg/min on a single t3.micro vCPU — roughly 3–5× what equivalent INSERT batches achieve.

**`unnest()` for parameterized bulk inserts:**

PostgreSQL supports array unnesting to send one round-trip with typed arrays, avoiding the N×parameter explosion of row-by-row inserts. This is not available in standard ANSI SQL:

```
INSERT INTO messages (message_id, user_id, room_id, username,
content, timestamp)
SELECT * FROM unnest(
    $1::uuid[], $2::text[], $3::text[], $4::text[], $5::text[],
    $6::timestamptz[]
) AS t(message_id, user_id, room_id, username, content, timestamp)
ON CONFLICT (message_id) DO NOTHING;
```

**`ON CONFLICT DO UPDATE` for atomic counter increments:**

Pre-aggregated stats tables use upsert to atomically accumulate counts without read-modify-write races. `date_trunc('minute', timestamp)` normalizes any timestamp to its minute boundary for time-bucket aggregation:

```
INSERT INTO message_stats (bucket, message_count)
VALUES (date_trunc('minute', $1), $2)
ON CONFLICT (bucket) DO UPDATE
    SET message_count = message_stats.message_count +
excluded.message_count;
```

## **TIMESTAMPTZ:**

Stores timestamps as UTC internally regardless of the inserting server's timezone, preventing silent data corruption when chat servers run in different regions.

## **Schema Design:**

```
-- Core message store (append-only event log)
CREATE TABLE messages (
  message_id  UUID          PRIMARY KEY,
  user_id     TEXT          NOT NULL,
  room_id     TEXT          NOT NULL,
  server_id   TEXT          NOT NULL,
  username    TEXT          NOT NULL,
  content     TEXT          NOT NULL,
  message_type TEXT        NOT NULL,
  timestamp   TIMESTAMPTZ  NOT NULL
);

-- Room participation tracking (upserted per batch, deduped at app layer)
CREATE TABLE user_rooms (
  user_id      TEXT          NOT NULL,
  room_id      TEXT          NOT NULL,
  last_activity_timestamp TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (user_id, room_id)
);
```

```
-- Pre-aggregated time-bucket stats (1-minute granularity)
CREATE TABLE message_stats (
  bucket      TIMESTAMPTZ PRIMARY KEY,
  message_count BIGINT DEFAULT 0
);

-- Pre-aggregated per-user message counts
CREATE TABLE user_message_stats (
  user_id     TEXT          PRIMARY KEY,
  message_count BIGINT DEFAULT 0
);

-- Pre-aggregated per-room message counts
CREATE TABLE room_message_stats (
  room_id     TEXT          PRIMARY KEY,
  message_count BIGINT DEFAULT 0
);
```

- messages is an append-only log. UUID PK silently rejects duplicate RabbitMQ deliveries — idempotent by construction with no extra application logic.
- user\_rooms composite PK (user\_id, room\_id) covers Q4 directly. Application-layer deduplication before each batch avoids redundant upsert conflicts and lock contention on the index.

- message\_stats, user\_message\_stats, room\_message\_stats are pre-aggregated counters updated via ON CONFLICT DO UPDATE. Analytics queries become O(1) scans on small tables instead of COUNT(\*) GROUP BY over tens of millions of rows.

### **Indexing Strategy:**

```
CREATE INDEX idx_messages_room_id_timestamp ON messages(room_id, timestamp DESC);
CREATE INDEX idx_messages_user_id ON messages(user_id, timestamp DESC);
CREATE INDEX idx_messages_timestamp ON messages(timestamp);
```

### **Scaling considerations:**

Vertical (immediate): The t3.micro consumer (1 vCPU, 1GB RAM) is the current bottleneck at ~2,500 msg/s average write rate. A t3.small (2 vCPU) is estimated to add 40–60% throughput by eliminating CPU contention between consumer goroutines, DB writers, and stats aggregators sharing one core.

Table partitioning (medium-term): At 3,200 msg/s sustained, messages grows ~280M rows/day. Declarative range partitioning by month keeps active partition size bounded, reduces index depth, and allows cheap partition drops for retention policy:

```
CREATE TABLE messages (...) PARTITION BY RANGE (timestamp);
CREATE TABLE messages_2026_03 PARTITION OF messages
    FOR VALUES FROM ('2026-03-01') TO ('2026-04-01');
```

Read replicas: Stream WAL to a standby for analytics queries — decouples the write path from reporting load with no schema changes required.

Connection pooling: Current pgxpool with MaxConns = 8 (DB\_WORKERS + STATS\_WORKERS + 2). For production with many app instances, PgBouncer in transaction mode multiplexes hundreds of client connections onto a small Postgres connection pool without exhausting max\_connections.

### **Backup and Recovery:**

Idempotent writes: message\_id UUID PK rejects duplicate deliveries silently. No special deduplication logic is needed on recovery — re-replaying RabbitMQ messages is safe.

Point-in-time recovery: WAL (Write-Ahead Logging) archiving to S3 (archive\_mode = on) provides continuous archiving with RPO(Recovery Time Objective) < 5 minutes. Combined with a nightly pg\_dump logical backup for fast restore of last-known-good state without WAL replay.

Sai Supriya Kotturu

For this assignment: Manual pg\_dump checkpoint before each test run:

```
pg_dump -Fc chatflow > chatflow_$(date +%Y%m%d_%H%M).dump
```

Dead letter queue: Failed DB batches (after circuit breaker opens) are redirected to an in-memory dlq\_chan and retried with exponential backoff (1s → 2s → 4s → 8s) once the circuit closes. No messages are silently dropped – worst case is delayed persistence, not data loss.

### System Architecture

```
Client (local) → ALB → 2x Chat Server t3.small (port 3000) → RabbitMQ t3.micro
                                                         ↓
                                                         Consumer-v3 t3.micro
                                                         ↓
                                                         PostgreSQL 16 (localhost)
```

### Consumer pipeline:

```
RabbitMQ persistence_queue
  → 5 consumer goroutines (unmarshal, fan-out)
    ↓ db_chan (buffer 10K)          ↓ stats_chan (non-blocking, drop ok)
  → 3 DB writer goroutines          → 3 stats aggregator goroutines
    ↓ CopyFrom batch (messages)      ↓ upsert (message_stats, user/room counters)
    ↓ upsert batch (user_rooms)
    ↓ circuit breaker → dlq_chan on failure
```

### Performance Report:

[https://github.khoury.northeastern.edu/supriyakotturu/chat-flow/blob/0fe19cd715e23a9f40ef707dac82e04ec6b9095f/docs/assignment3/PERFORMANCE\\_REPORT.md](https://github.khoury.northeastern.edu/supriyakotturu/chat-flow/blob/0fe19cd715e23a9f40ef707dac82e04ec6b9095f/docs/assignment3/PERFORMANCE_REPORT.md)

### Configuration Files:

<https://github.khoury.northeastern.edu/supriyakotturu/chat-flow/blob/0fe19cd715e23a9f40ef707dac82e04ec6b9095f/docs/assignment3/RESULTS.md>

<https://github.khoury.northeastern.edu/supriyakotturu/chat-flow/blob/0fe19cd715e23a9f40ef707dac82e04ec6b9095f/load-tests/config.md>

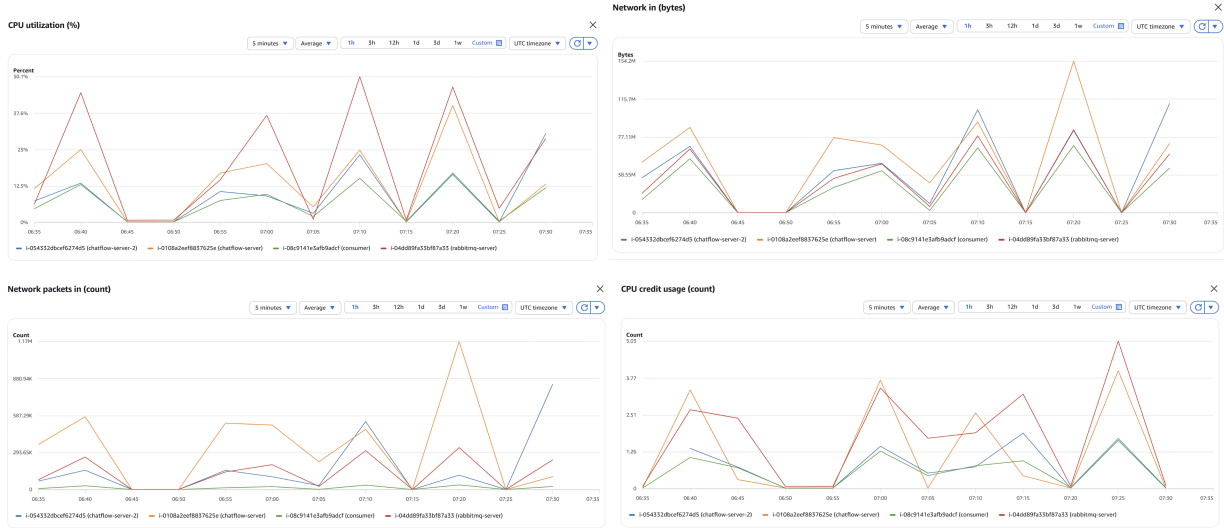
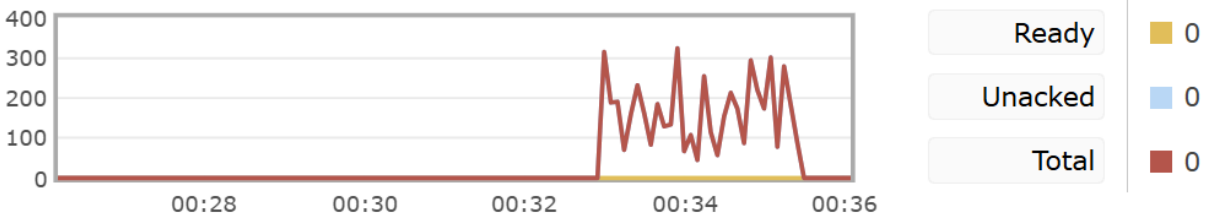
### Detailed Test Results:

<https://github.khoury.northeastern.edu/supriyakotturu/chat-flow/blob/assignment-3/load-tests/Chatflow%20assignment-3%20test%20results.pdf>

**BATCH\_SIZE=5000, FLUSH\_INTERVAL=1000ms**

152.7s

Queued messages last ten minutes ?



```
=== Performance Metrics ===
Total runtime (wall time): 152.7s
Successful messages sent: 500000
Failed messages: 0
Overall throughput: 3273.9 msg/s
Total connections: 1000
Reconnections: 24000
Failed connections: 0
Mean Latency across all rooms: 379ms
Median Latency across all rooms: 374ms
95th Percentile Latency across all rooms: 462ms
99th Percentile Latency across all rooms: 485ms
Min Latency across all rooms: 214ms
Max Latency across all rooms: 502ms
Median Throughput across all rooms: 748.54 msg/s
Throughput across rooms:

Room 9 | users: 50 | throughput: 37088.2 msg/s | mean latency: 387ms | median latency: 394ms
Message Types: JOIN: 36943 TEXT: 668078 LEAVE: 37040

Room 17 | users: 50 | throughput: 33655.0 msg/s | mean latency: 415ms | median latency: 417ms
Message Types: TEXT: 600659 LEAVE: 33319 JOIN: 33269

Room 18 | users: 50 | throughput: 38985.4 msg/s | mean latency: 354ms | median latency: 358ms
Message Types: TEXT: 667129 LEAVE: 36808 JOIN: 36527

Room 14 | users: 50 | throughput: 40173.7 msg/s | mean latency: 348ms | median latency: 348ms
Message Types: TEXT: 702414 LEAVE: 38809 JOIN: 38673

Room 11 | users: 50 | throughput: 39085.2 msg/s | mean latency: 361ms | median latency: 362ms
Message Types: LEAVE: 36836 JOIN: 36522 TEXT: 668459

Room 7 | users: 50 | throughput: 34083.5 msg/s | mean latency: 347ms | median latency: 345ms
Message Types: JOIN: 31874 TEXT: 576202 LEAVE: 31937

Room 10 | users: 50 | throughput: 36415.7 msg/s | mean latency: 430ms | median latency: 431ms
Message Types: LEAVE: 36145 JOIN: 35940 TEXT: 658312

Room 2 | users: 50 | throughput: 38783.8 msg/s | mean latency: 350ms | median latency: 349ms
Message Types: TEXT: 600085 LEAVE: 36298 JOIN: 36205

Room 16 | users: 50 | throughput: 35193.8 msg/s | mean latency: 441ms | median latency: 441ms
Message Types: TEXT: 652699 LEAVE: 36147 JOIN: 35804
```

## Sai Supriya Kotturu

```
Room 8 | users: 50 | throughput: 24513.9 msg/s | mean latency: 382ms | median latency: 393ms
Message Types: TEXT: 420940 JOIN: 23291 LEAVE: 23088

Room 15 | users: 50 | throughput: 39855.4 msg/s | mean latency: 353ms | median latency: 356ms
Message Types: LEAVE: 37270 JOIN: 37088 TEXT: 678719

Room 19 | users: 50 | throughput: 36695.9 msg/s | mean latency: 359ms | median latency: 362ms
Message Types: LEAVE: 35718 JOIN: 35620 TEXT: 644523

Room 20 | users: 50 | throughput: 35540.5 msg/s | mean latency: 386ms | median latency: 387ms
Message Types: LEAVE: 33080 JOIN: 32804 TEXT: 603498

Room 13 | users: 50 | throughput: 37063.5 msg/s | mean latency: 391ms | median latency: 391ms
Message Types: JOIN: 36022 TEXT: 656965 LEAVE: 36163

Room 1 | users: 50 | throughput: 44813.2 msg/s | mean latency: 236ms | median latency: 234ms
Message Types: JOIN: 28471 LEAVE: 28479 TEXT: 513066

Room 4 | users: 50 | throughput: 34224.8 msg/s | mean latency: 356ms | median latency: 357ms
Message Types: LEAVE: 31471 JOIN: 31286 TEXT: 571052

Room 12 | users: 50 | throughput: 37419.1 msg/s | mean latency: 370ms | median latency: 372ms
Message Types: LEAVE: 34884 JOIN: 34690 TEXT: 630566

Room 6 | users: 50 | throughput: 37135.8 msg/s | mean latency: 409ms | median latency: 411ms
Message Types: LEAVE: 33502 JOIN: 33193 TEXT: 607924

Room 5 | users: 50 | throughput: 37649.5 msg/s | mean latency: 449ms | median latency: 449ms
Message Types: LEAVE: 34992 JOIN: 36941 TEXT: 654204

Room 3 | users: 50 | throughput: 38000.2 msg/s | mean latency: 453ms | median latency: 454ms
Message Types: TEXT: 681677 LEAVE: 36245 JOIN: 38623

----- Consumer Metrics -----
```

```
--- Core ---
Active users (last 1h): 50
Busiest room:          13 (25000 messages)
Most active user:      39 (20 rooms)
  room 1                last active: 2026-03-25T07:35:18.687749Z
  room 6                last active: 2026-03-25T07:35:15.264347Z
  room 16               last active: 2026-03-25T07:35:04.690407Z
  room 13               last active: 2026-03-25T07:34:59.138897Z
  room 20               last active: 2026-03-25T07:34:47.826457Z
  room 18               last active: 2026-03-25T07:34:40.174925Z
  room 19               last active: 2026-03-25T07:34:35.181111Z
  room 12               last active: 2026-03-25T07:34:30.045454Z
  room 2                last active: 2026-03-25T07:34:20.989464Z
  room 10               last active: 2026-03-25T07:34:08.066565Z
  room 17               last active: 2026-03-25T07:34:06.723341Z
  room 9                last active: 2026-03-25T07:34:06.000832Z
  room 7                last active: 2026-03-25T07:33:51.19318Z
  room 11               last active: 2026-03-25T07:33:41.50871Z
  room 4                last active: 2026-03-25T07:33:30.466422Z
  room 14               last active: 2026-03-25T07:33:25.930452Z
  room 15               last active: 2026-03-25T07:33:25.858004Z
  room 3                last active: 2026-03-25T07:33:09.817484Z
  room 8                last active: 2026-03-25T07:33:04.341719Z
  room 5                last active: 2026-03-25T07:33:04.13381Z
```

```
--- Messages/min (last 15 buckets) ---  
2026-03-25T07:35:00Z 61109 msg  
2026-03-25T07:34:00Z 197594 msg  
2026-03-25T07:33:00Z 198718 msg  
2026-03-25T07:32:00Z 42579 msg
```

```
--- Top 5 users by message count ---  
1. 44 - 10000 messages
```

```
--- Top 5 users by message count ---  
1. 44 - 10000 messages  
2. 25 - 10000 messages  
3. 32 - 10000 messages  
4. 41 - 10000 messages  
5. 39 - 10000 messages
```

```
--- Top 5 rooms by message count ---  
1. 2 - 25000 messages  
2. 8 - 25000 messages  
3. 7 - 25000 messages  
4. 4 - 25000 messages  
5. 13 - 25000 messages
```

```
--- Top 5 users by room count ---  
1. 8 - 20 rooms  
2. 12 - 20 rooms  
3. 10 - 20 rooms  
4. 7 - 20 rooms  
5. 49 - 20 rooms
```

```
=====
```

```
→ chat-flow git:(assignment-3) 🌸 🍄 🎈
```